COMP-SCI-5565
Intro to Statistical Learning

Topic 6 – Tree-based Methods

Adu Baffour, PhD

University of Missouri-Kansas City
Division of Computing, Analytics, and Mathematics
School of Science and Engineering
aabnxq@umkc.edu

UMKC

Lecture Objectives

- Use basic decision trees to model relationships between predictors and an outcome.
- Compare and contrast tree-based models with other model types.
- Use tree-based ensemble methods to build (better!) predictive models.
- Compare and contrast the various methods of building tree ensembles: bagging, boosting, and random forests.

Outline

6.1 Introduction

6.2 Regression Trees

6.3 Regression Trees Building Process

6.4 Classification Trees

6.5 Advantages/Disadvantages of Decision Trees

6.6 Bagging

6.7 Random Forest

6.8 Boosting

Outline

6.1 Introduction

6.2 Regression Trees

6.3 Regression Trees Building Process

6.4 Classification Trees

6.5 Advantages/Disadvantages of Decision Trees

6.6 Bagging

6.7 Random Forest

6.8 Boosting

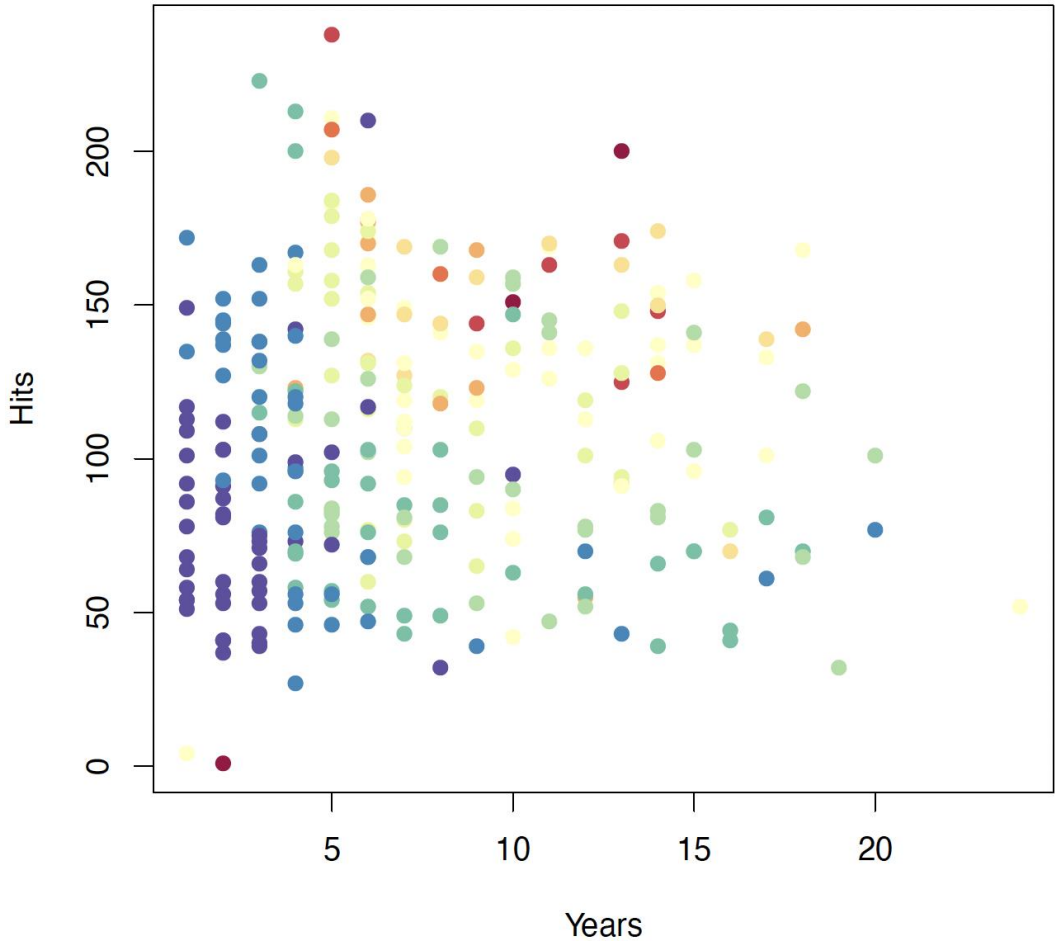
6.1 Introduction

- It involves stratifying or segmenting the predictor space into many simple regions.
- They are simple and useful for interpretation.
- However, basic decision trees are NOT competitive with the best supervised learning approaches regarding prediction accuracy.
- Thus, we also discuss *bagging*, *random forests*, and *boosting* (i.e., tree-based ensemble methods) to grow multiple trees, which are combined to yield a single consensus prediction.
- These can result in dramatic improvements in prediction accuracy (but some loss of interpretability).
- Decision trees can be applied to both regression and classification.

Outline

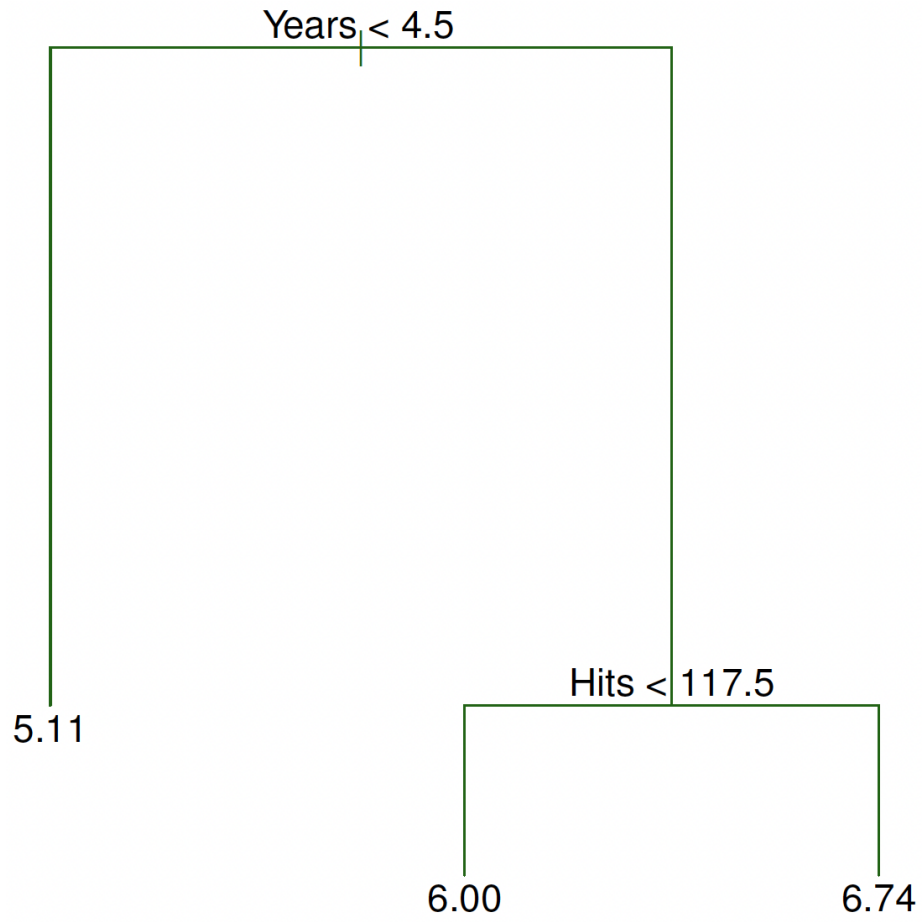
6.2 Regression Trees

Salary is color-coded from low (blue, green) to high (yellow, red)



6.2 Regression Trees

Decision tree for these data

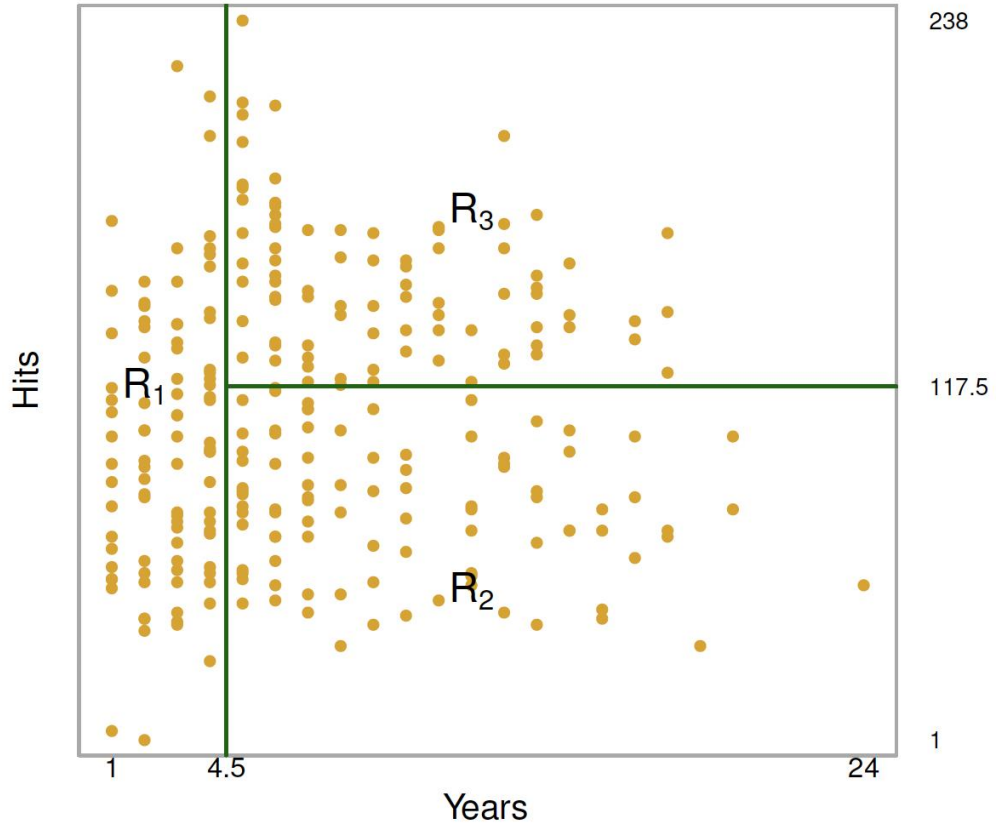


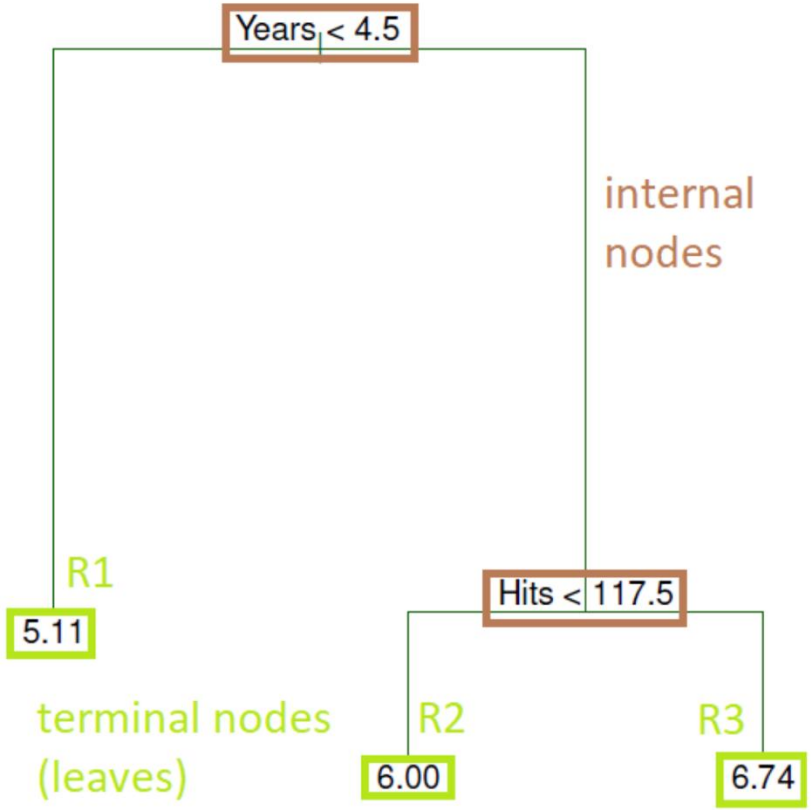
6.2 Regression Trees

- At a given internal node, the label (of the form $X_j < t_k$) indicates the left-hand branch emanating from that split, and the right-hand branch corresponds to $X_j \geq t_k$.
- For instance, the split at the top of the tree results in two large branches.
- The left-hand branch corresponds to $\text{Years} < 4.5$, and the right-hand branch corresponds to $\text{Years} \geq 4.5$.
- The tree has two internal nodes and three terminal nodes, or leaves.
- The number in each leaf is the mean of the response for the observations that fall there.

6.2 Regression Trees

- Overall, the tree stratifies or segments the players into three regions of predictor space: $R_1 = \{\mathrm{X} \mid \mathrm{Years} < 4.5\}$, $R_2 = \{\mathrm{X} \mid \mathrm{Years} \geq 4.5, \mathrm{Hits} < 117.5\}$, and $R_3 = \{\mathrm{X} \mid \mathrm{Years} \geq 4.5, \mathrm{Hits} \geq 117.5\}$.



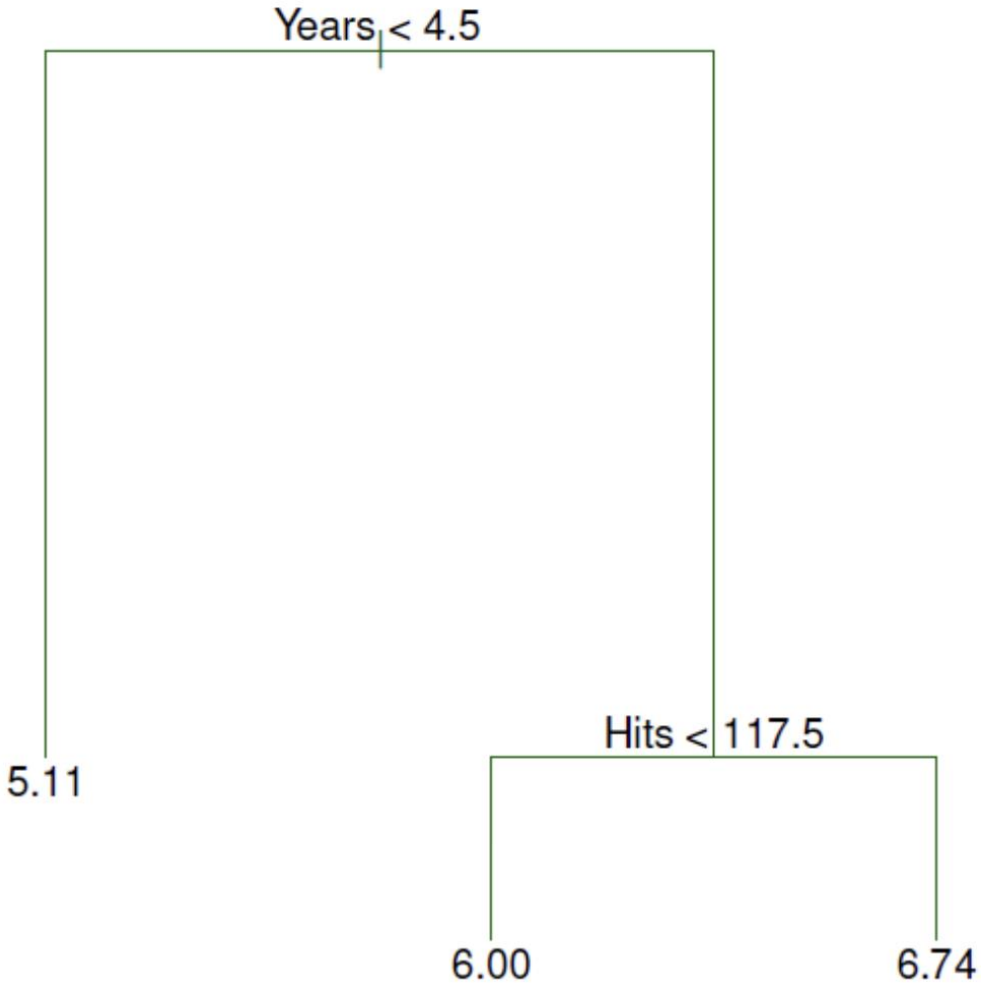


UMKC

6.2 Regression Trees

Interpretation of results: regression tree (Hitters data)

- Years is the most important factor in determining Salary: players with less experience earn lower salaries than more experienced players.
- Given that a player is less experienced, the number of Hits that he made in the previous year seems to play little role in his Salary.
- But among players who have been in the major leagues for 5 or more years, the number of Hits made in the previous year does affect Salary: players who made more Hits last year tend to have higher salaries
- This is surely an oversimplification, but compared to a regression model, it is easy to display, interpret, and explain.



UMKC

Outline

UMKC

6.3 Regression Tree Building Process

1. Divide the predictor space — that is, the set of possible values for $X_1, X_2, \ldots, X_p$ - into J distinct regions $R_1, R_2, \ldots, R_J$.
   - Regions can have ANY shape - they don't have to be boxes.
2. For every observation that falls into the region R_J, we make the same prediction: the mean of the response values in R_J.
3. The goal is to find regions (here boxes) $R_1, R_2, \ldots, R_J$ that minimize the RSS, given by

$$RSS = \sum_{i=1}^{J} \sum_{i \in R_J} \left(y_i - \hat{y}_{R_J}\right)^2$$

where $\hat{y}_{R_J}$ is the mean response for the training observations within the jth box.

- Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes.

UMKC

6.3 Regression Tree Building Process

Recursive Binary Splitting

- Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes.
- For this reason, we take a top-down, greedy approach known as *recursive binary splitting*.
- The approach is top-down because it begins at the top of the tree and then successively splits the predictor space; each split is indicated via two new branches further down on the tree.
- It is greedy because, at each step of the tree-building process, the best split is done at that particular step rather than looking ahead and picking a split that will lead to a better tree in some future step.

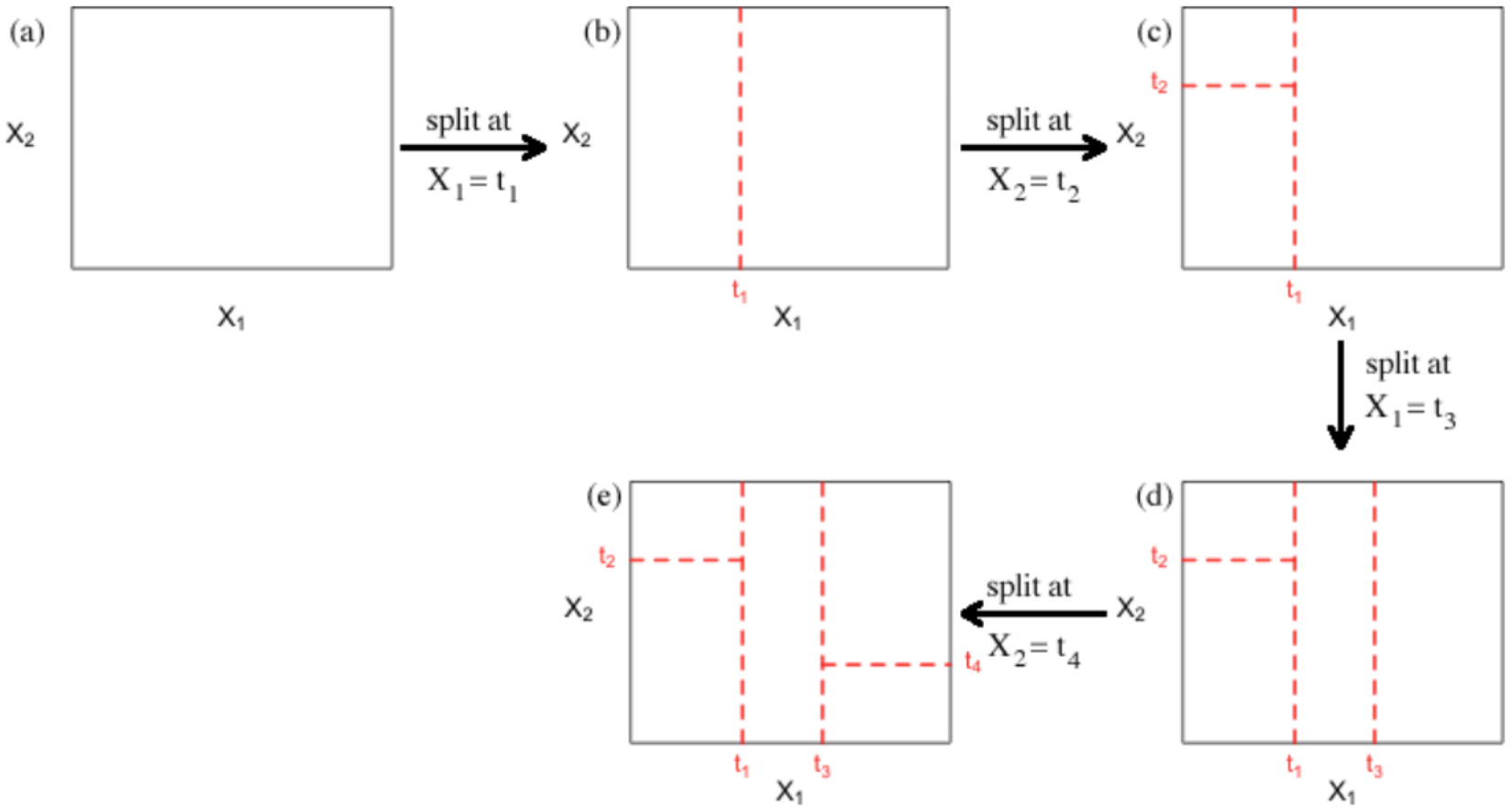
6.3 Regression Tree Building Process

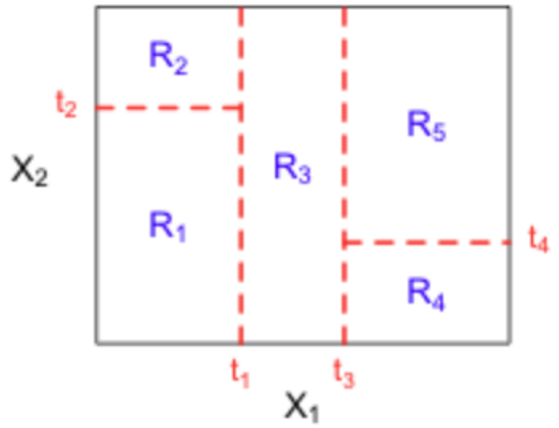
Recursive Binary Splitting – continued

1. We first select the predictor X_j and the cutpoint s such that splitting the predictor space into the regions $\{X|X_j < s\}$ and $\{X|X_j \geq s\}$ leads to the greatest possible reduction in RSS.
2. Repeat the process looking for the best predictor and best cutpoint to split data further (i.e., split one of the 2 previously identified regions - not the entire predictor space) minimizing the RSS within each of the resulting regions.
3. Continue until a stopping criterion is reached, e.g., no region contains more than five observations
4. Again, we predict the response for a given test observation using the mean of the training observations in the region to which that test observation belongs.

6.3 Regression Tree Building Process

Recursive Binary Splitting – continued

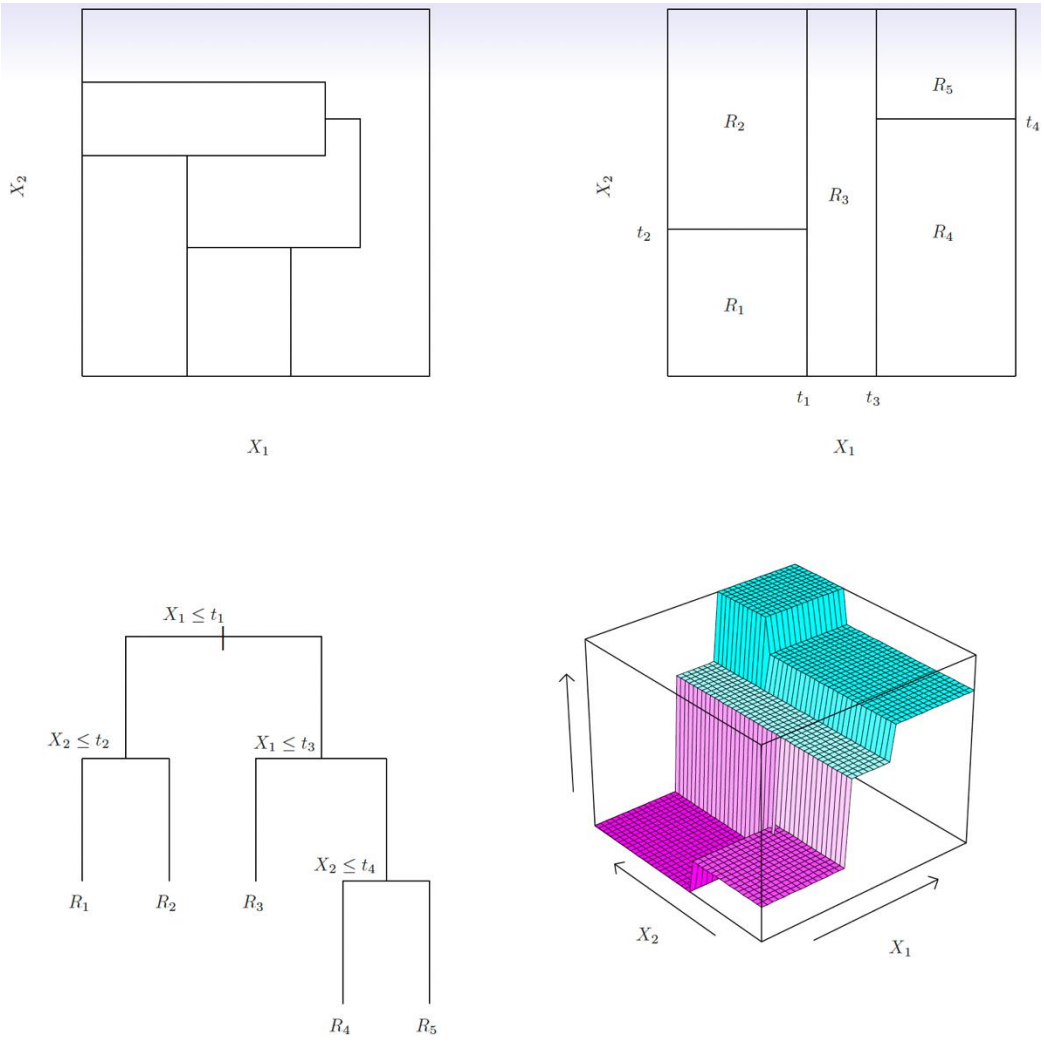




6.3 Regression Tree Building Process

Recursive Binary Splitting – continued

- Top Left: A partition of two-dimensional feature space that could not result from recursive binary splitting.
- Top Right: The output of recursive binary splitting on a two-dimensional example.
- Bottom Left: A tree corresponding to the partition in the top right panel.
- Bottom Right: A perspective plot of the prediction surface corresponding to that tree.



6.3 Regression Tree Building Process

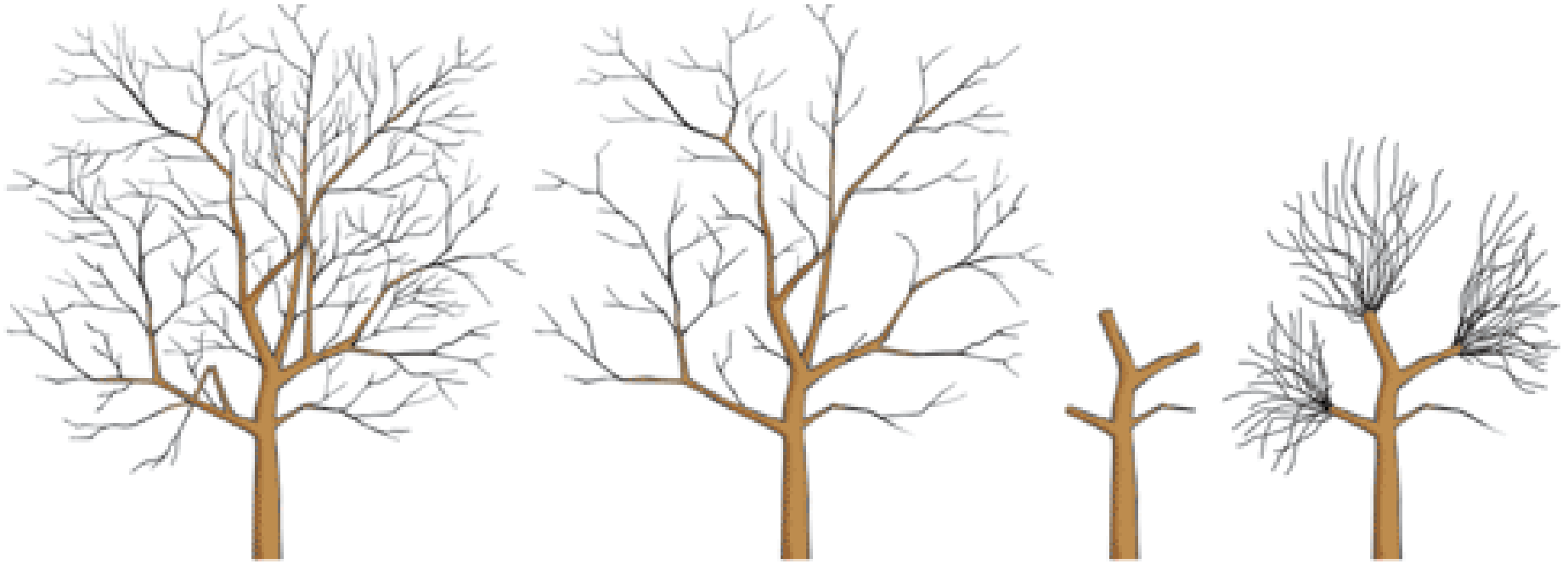
Recursive Binary Splitting – continued

- The process described above may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance. Why?
- The tree is too leafy (complex)
- A smaller tree with fewer splits (that is, fewer regions $R_1, R_2, \ldots, R_J$) might lead to lower variance and better interpretation at the cost of a little bias.
- So we will prune.

6.3 Regression Tree Building Process

Pruning a tree

- We grow a very large tree T_0 and prune it back in order to obtain a subtree.
- We use *cost complexity pruning* also known as weakest link pruning.



6.3 Regression Tree Building Process

Pruning a tree – continued

- We consider a sequence of trees indexed by a nonnegative tuning parameter α.
- For each value of α , there corresponds a subtree $\mathrm{T} \subset T_0$ that minimizes

$$RSS = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T|$$

where $\alpha > 0$, $|T|$ indicates the number of terminal nodes of the tree T, R_m is the rectangle (i.e. the subset of predictor space) corresponding to the mth terminal node, and $\hat{y}_{R_m}$ is the mean of the training observations in R_m.

UMKC

6.3 Regression Tree Building Process

Pruning a tree – continued

$$RSS = \sum_{m=1}^{|T|} \sum_{x_i \in R_m} \left(y_i - \hat{y}_{R_m}\right)^2 + \alpha|T|$$

- The tuning parameter α controls a trade-off between the subtree's complexity and its fit to the training data.
- We select an optimal value $\hat{\alpha}$ using cross-validation.
- We then return to the full data set and obtain the subtree corresponding to $\hat{\alpha}$.

6.3 Regression Tree Building Process

Pruning a tree summary

(1) Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum observations.

(2) Apply cost complexity pruning to the large tree to obtain a sequence of best subtrees as a function of α.

(3) Use K-fold cross-validation to choose α. For each $k = 1, \ldots, K$:

> 3.1 Repeat Steps 1 and 2 for each the $K - 1/K$th fraction of the training data.
>
> 3.2 Average the results and pick α to minimize the average MSE.

(4) Return the subtree from Step 2 that corresponds to the chosen value of α.

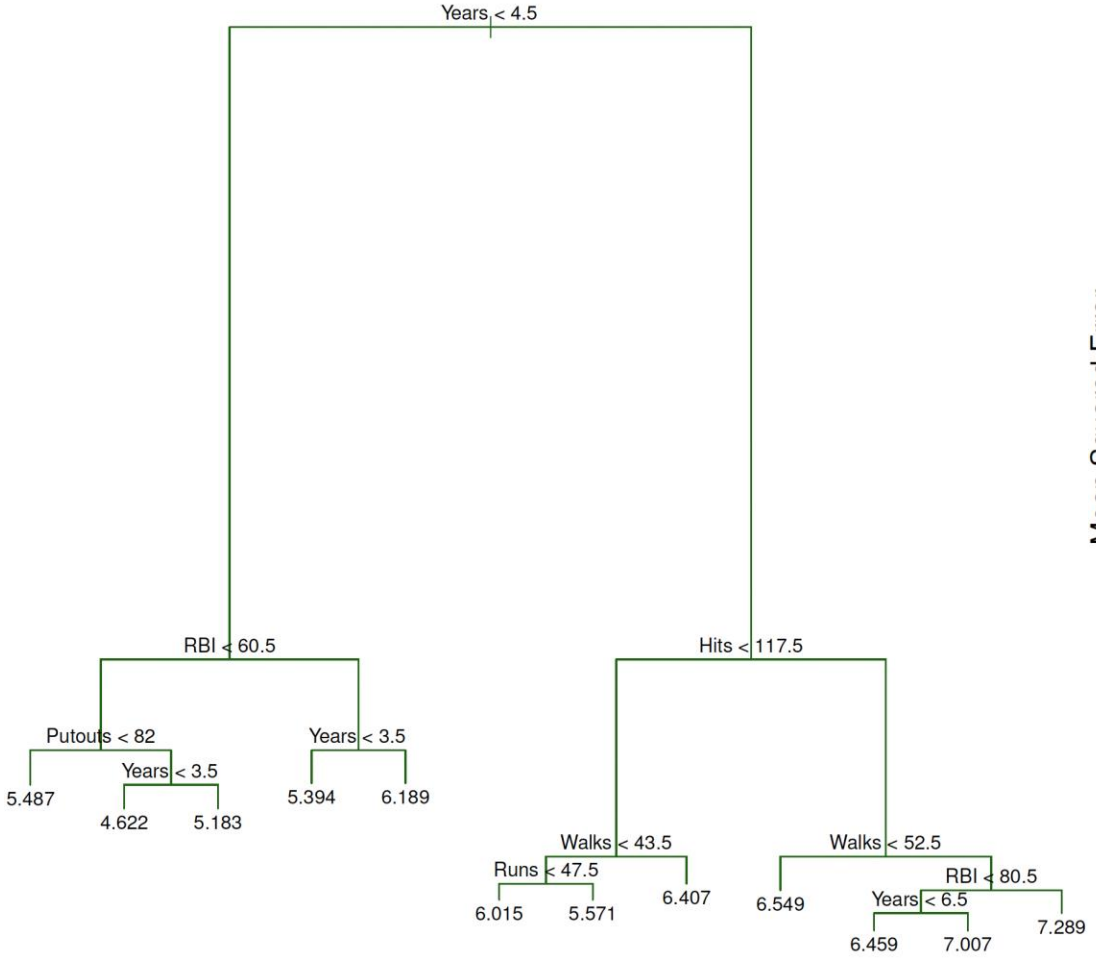
6.3 Regression Tree Building Process

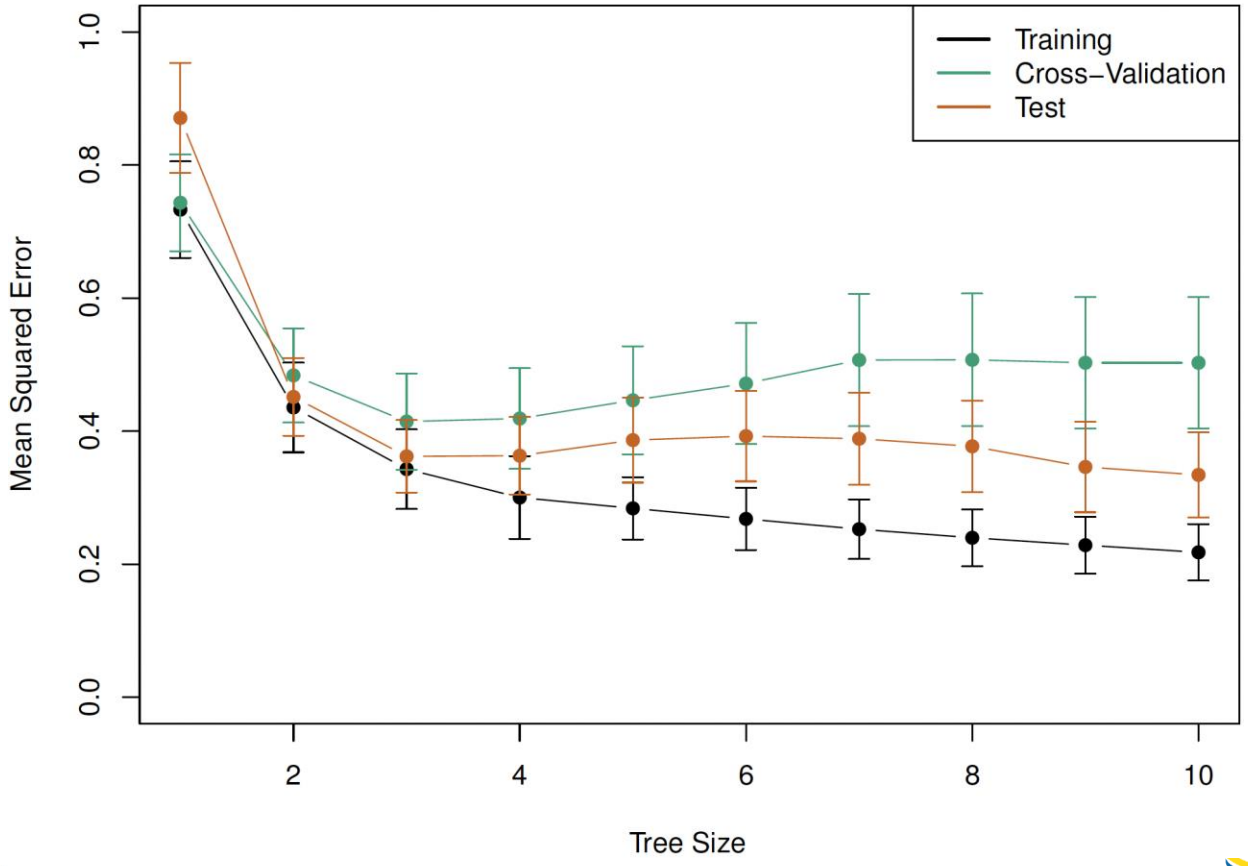
Pruning a tree example (Hitters data)

- First, we randomly divided the data set in half, yielding 132 observations in the training set and 131 in the test set.
- We then built a large regression tree on the training data and varied α to create subtrees with different numbers of terminal nodes.
- Finally, we performed six-fold cross-validation to estimate the cross-validated MSE of the trees as a function of α.
- Training, cross-validation, and test MSE are shown as a function of the number of terminal nodes in the pruned tree.
- Standard error bands are displayed. The minimum cross-validation error occurs at a tree size of 3.

6.3 Regression Tree Building Process

Pruning a tree example (Hitters data)





Outline

6.1 Introduction

6.2 Regression Trees

6.3 Regression Trees Building Process

6.4 Classification Trees

6.5 Advantages/Disadvantages of Decision Trees

6.6 Bagging

6.7 Random Forest

6.8 Boosting

6.4 Classification Trees

- It is like a regression tree, except it predicts a qualitative (vs quantitative) response.
- We predict that each observation belongs to the most commonly occurring class of training observations in the region to which it belongs.
- RSS cannot be used as a criterion for making the binary splits in the classification setting.
- A natural alternative to RSS is the classification error rate, i.e., the fraction of the training observations in that region that do not belong to the most common class:

$$E = 1 - \max_k(\hat{p}_{mk})$$

where $\hat{p}_{mk}$ is the proportion of training observations in the mth region that are from the kth class.

- However, this error rate is unsuited for tree-based classification because E does not change much as the tree grows (lacks sensitivity), in practice two other measures are preferable.

UMKC

6.4 Classification Trees

Gini Index

- The Gini index, defined by

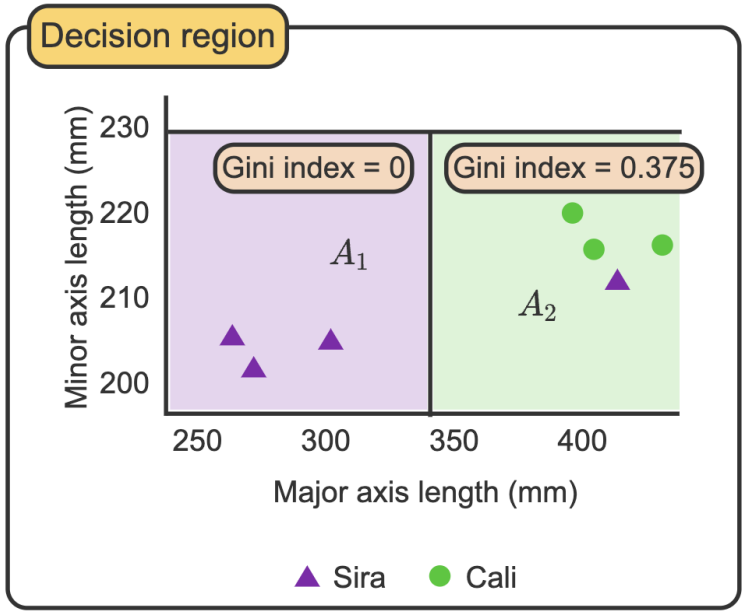
$$G = 1 - \sum_{k=1}^{n} (\hat{p}_{mk})^2$$

is a measure of total variance across the K classes.

- The Gini index takes on a small value if all of the $\hat{p}_{mk}$ are close to zero or one.
- For this reason, the Gini index is referred to as a measure of node impurity – a small value indicates that a node contains predominantly observations from a single class.

6.4 Classification Trees

- A subset of dry beans with minor axis length less than 230 mm is split into two regions, A1 and A2.
- The region on the left,A1, contains only one class, so the Gini index is 0. Here, the split is pure, so no calculation is necessary.
- The region on the right, A2, contains 1 Sira bean and 3 Cali beans. The Gini index for A2 is 0.375.
- The overall Gini index for the entire model is the weighted average of the Gini indices for all splits. Here, the overall Gini index is 0.214.



Gini index for A_2

$$\text{Gini index} = 1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 = 0.375$$

Overall Gini index for the model

$$\text{Gini index} = \frac{3}{7}(0) + \frac{4}{7}(0.375) = 0.214$$

UMKC

6.4 Classification Trees

Cross-entropy

- An alternative to the Gini index is cross-entropy, given by

$$\mathrm{D} = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk} .$$

- The Gini index and cross-entropy are very similar numerically.

UMKC

6.4 Classification Trees

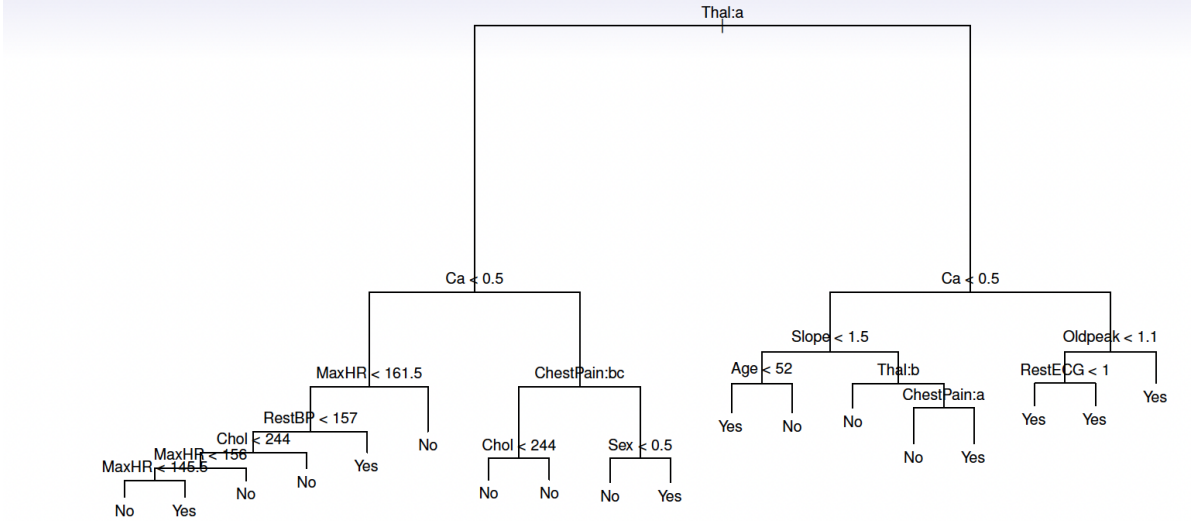
Example: classification tree (Heart data)

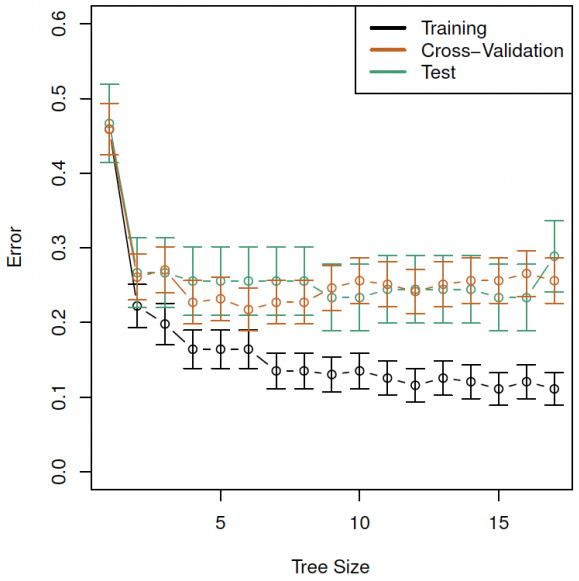
- Data contain a binary outcome HD (heart disease Y or N based on angiographic test) for 303 patients who presented with chest pain.
- 13 predictors, including Age, Sex, Chol (cholesterol measurement), and other heart and lung function measurements.
- Cross-validation yields a tree with six terminal nodes.

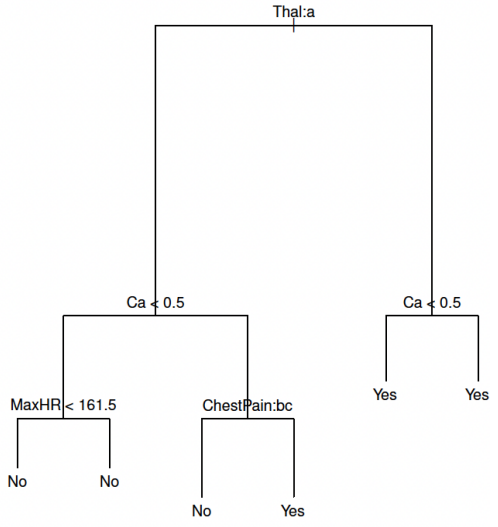
6.4 Classification Trees

Example: classification tree (Heart data)

- Data contain a binary outcome HD (heart disease Y or N based on angiographic test) for 303 patients who presented with chest pain.
- 13 predictors, including Age, Sex, Chol (cholesterol measurement), and other heart and lung function measurements.
- Cross-validation yields a tree with six terminal nodes.
- Even though the split RestECG<1 does not reduce the classification error, it improves the Gini index and the entropy, which are more sensitive to node purity.







Outline

UMKC

6.5 Advantages/Disadvantages of decision trees

- Trees can be displayed graphically and are very easy to explain to people.
- They mirror human decision-making.
- Can handle qualitative predictors without the need for dummy variables.

BUT,

- They do not have the same level of predictive accuracy.
- It can be very non-robust (i.e., a small change in the data can cause a large change in the final estimated tree).
- To improve performance, we can use an ensemble method combining many simple ‘building blocks’ (i.e., regression or classification trees) to obtain a single and potentially very powerful model.
- Ensemble methods include bagging, random forests, boosting, and Bayesian additive regression trees.

Outline

6.6 Bagging

- Bootstrap aggregation, or bagging, is a general-purpose procedure for reducing the variance of a statistical learning method.
- It is useful and frequently used in the context of decision trees.
- Recall that given a set of n independent observations $Z_1, \ldots, Z_n$, each with variance σ^2, the variance of the mean Z of the observations is given by σ^2/n.
- In other words, averaging a set of observations reduces variance.
- Of course, this is not practical because we generally do not have access to multiple training sets.
- What do we do?

6.6 Bagging

- We can bootstrap by taking repeated samples from the (single) training data set.
- Generate B different bootstrapped training data sets.
- Then train our method on the bth bootstrapped training set to get $\hat{f}^{*b}$, the prediction at a point x.
- Average all the predictions to obtain

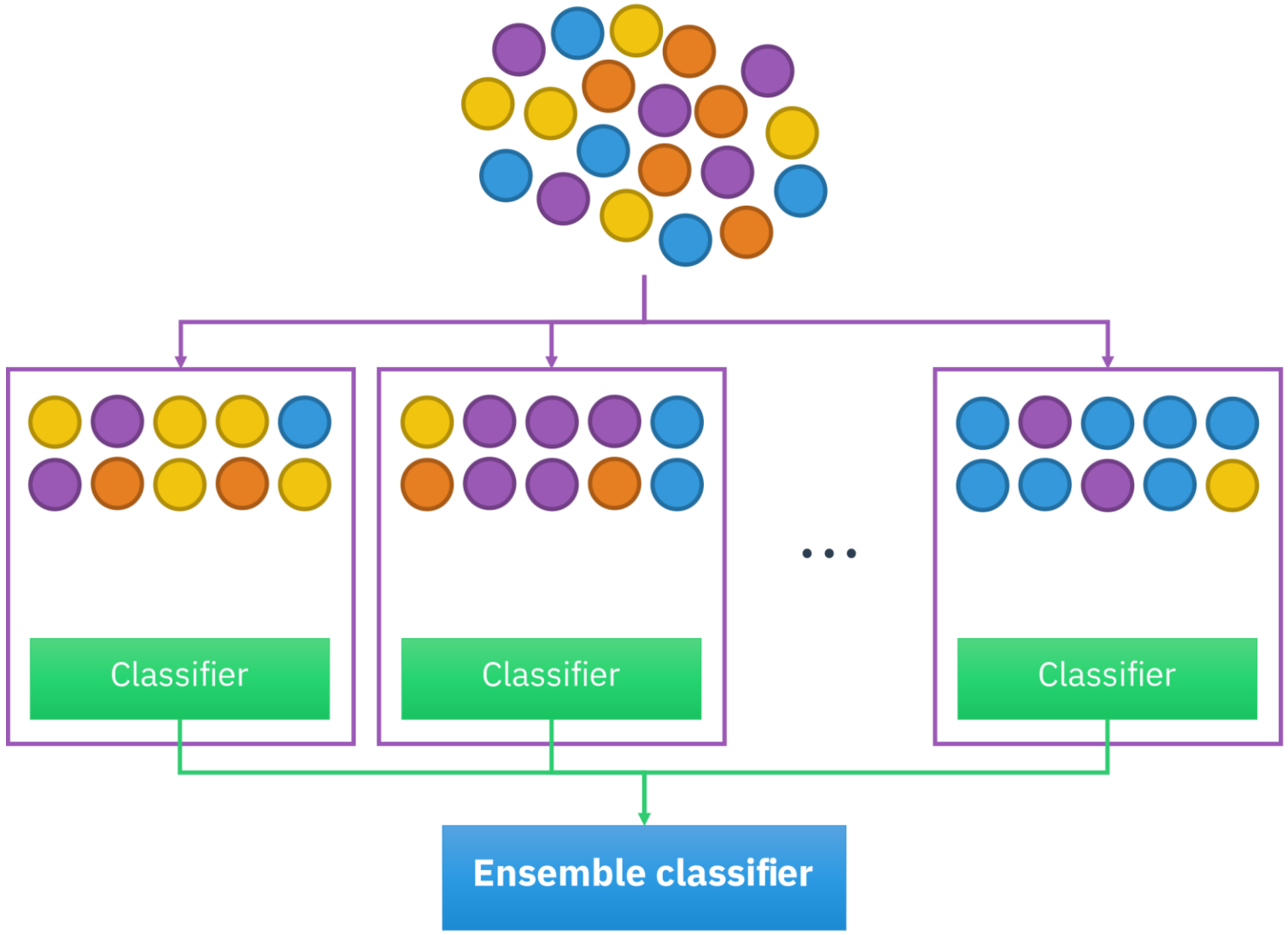
$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}^{*b}(x)$$

- This is called *bagging*.

6.6 Bagging

- The above prescription is applied to regression trees.
- In the case of classification trees:
  - For each test observation:
    - Record the class predicted by each of the B trees.
    - Take a *majority vote*: the overall prediction is the most commonly occurring class among the B predictions.
- NOTE: the number of trees B is not a critical parameter with bagging - a large B will not lead to overfitting

6.6 Bagging



Original Data

Bootstrapping

Aggregating

Bagging

UMKC

6.6 Bagging

- But how do we estimate the test error of a bagged model?
- It's straightforward:
    1. Because trees are repeatedly fit to bootstrapped subsets of observations, each bagged tree uses about 2/3 of the observations on average.
    2. The leftover 1/3 not used to fit a given bagged tree are called *out-of-bag* (OOB) observations.
    3. We can predict the response for the ith observation using each of the trees in which that observation was OOB. It gives around $B/3$ predictions for the ith observation (which we then average).

Outline

6.7 Random Forests

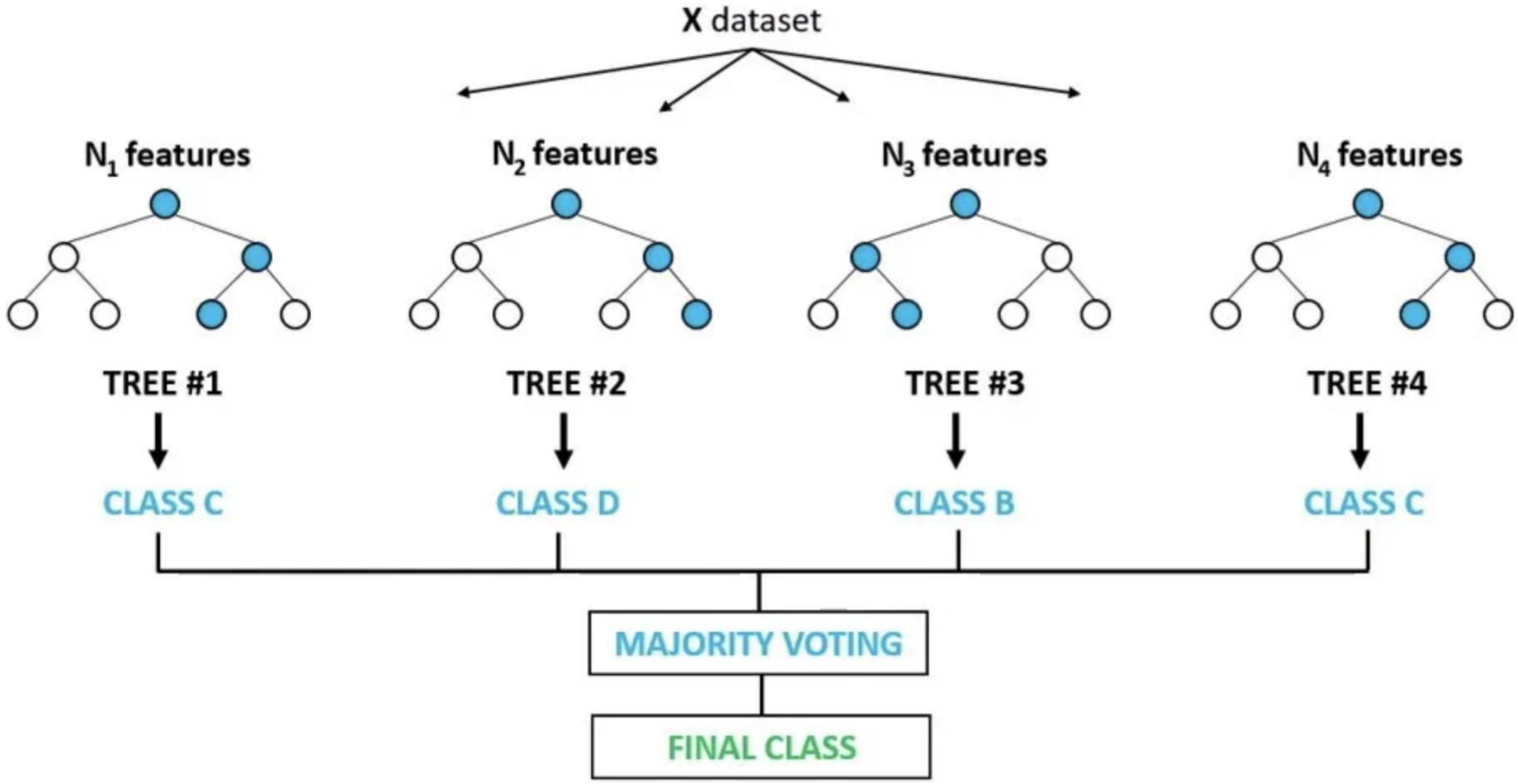
- A problem with bagging is that bagged trees may be highly similar.
- For example, if there is a strong predictor in the data set, most of the bagged trees will use this strong predictor in the top split so that
  - the trees will look quite similar
  - predictions from the bagged trees will be highly correlated.
- Averaging many highly correlated quantities does not lead to as large a reduction in variance as averaging many uncorrelated quantities.

6.7 Random Forests

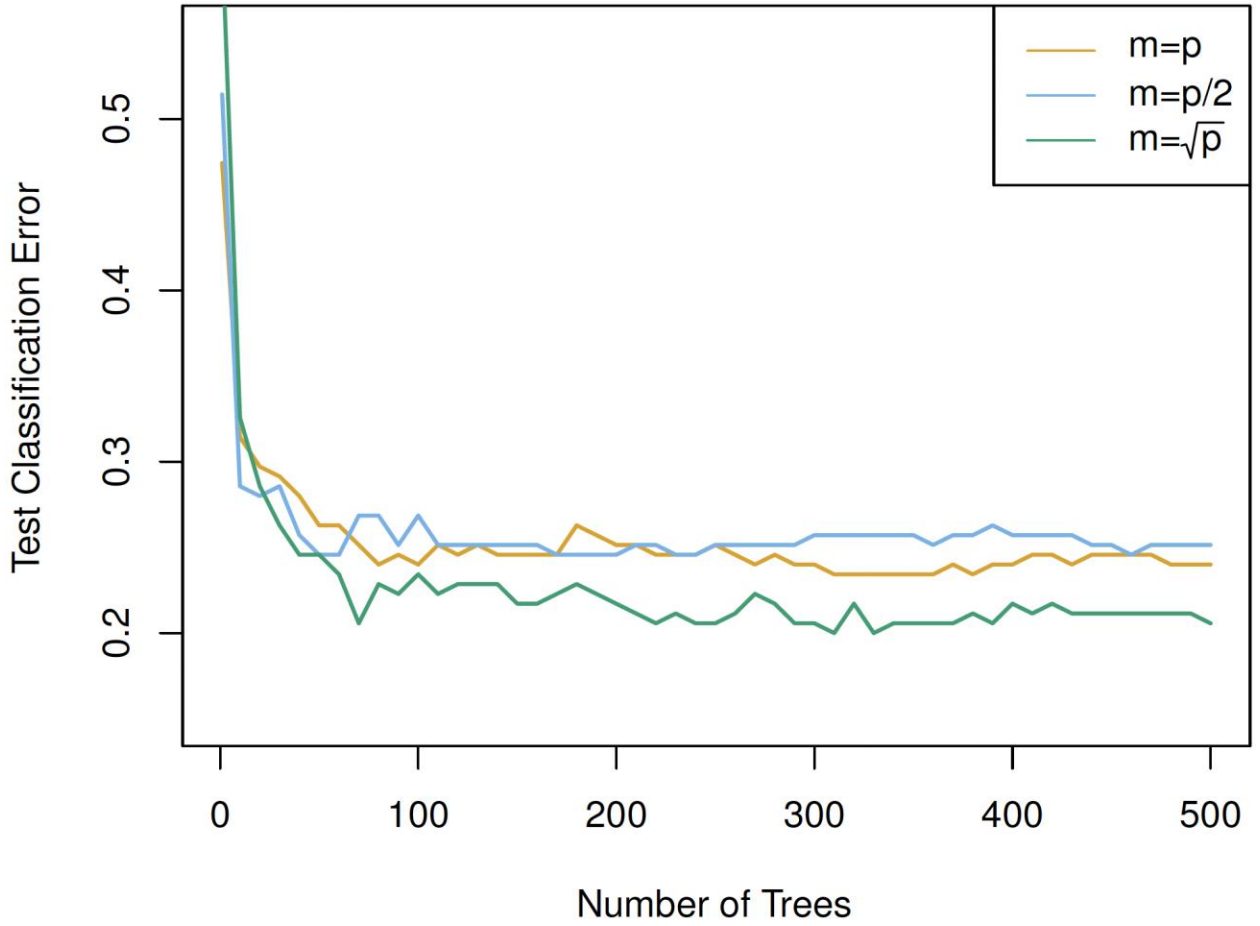
- Random forests overcome this problem by forcing each split to consider only a subset of the predictors (typically a random sample $m \approx \sqrt{p}$).
- Thus, at each split, the algorithm is NOT ALLOWED to consider a majority of the available predictors.
- This *decorrelates* the trees and makes the average of the resulting trees less variable (more reliable).
- Only difference between bagging and random forests is the choice of predictor subset size m at each split: if a random forest is built using $m = \mathrm{p}$ that's just bagging.
- For both, we build a number of decision trees on bootstrapped training samples.

UMKC

6.7 Random Forests



6.7 Random Forests



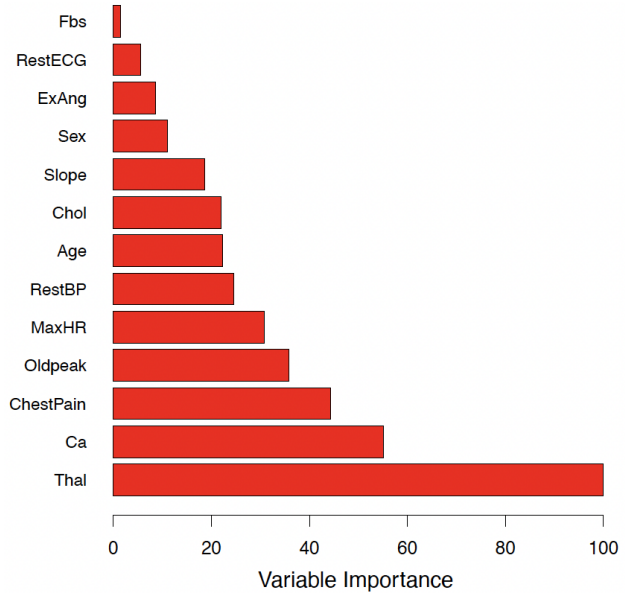
Results from random forests for the 15-class gene expression data set with $p = 500$ predictors. The test error is displayed as a function of the number of trees. Random forests ($m < p$) lead to a slight improvement over bagging ($m = p$). A single classification tree has an error rate of 45.7%.

UMKC

6.7 Random Forest

Variable importance measures

- For bagged/RF regression trees, we record the total amount that the RSS is decreased due to splits over a given predictor, averaged over all B trees. A large value indicates an important predictor.
- Similarly, for bagged/RF classification trees, we add up the total amount that the Gini index is decreased by splits over a given predictor, averaged over all B trees.



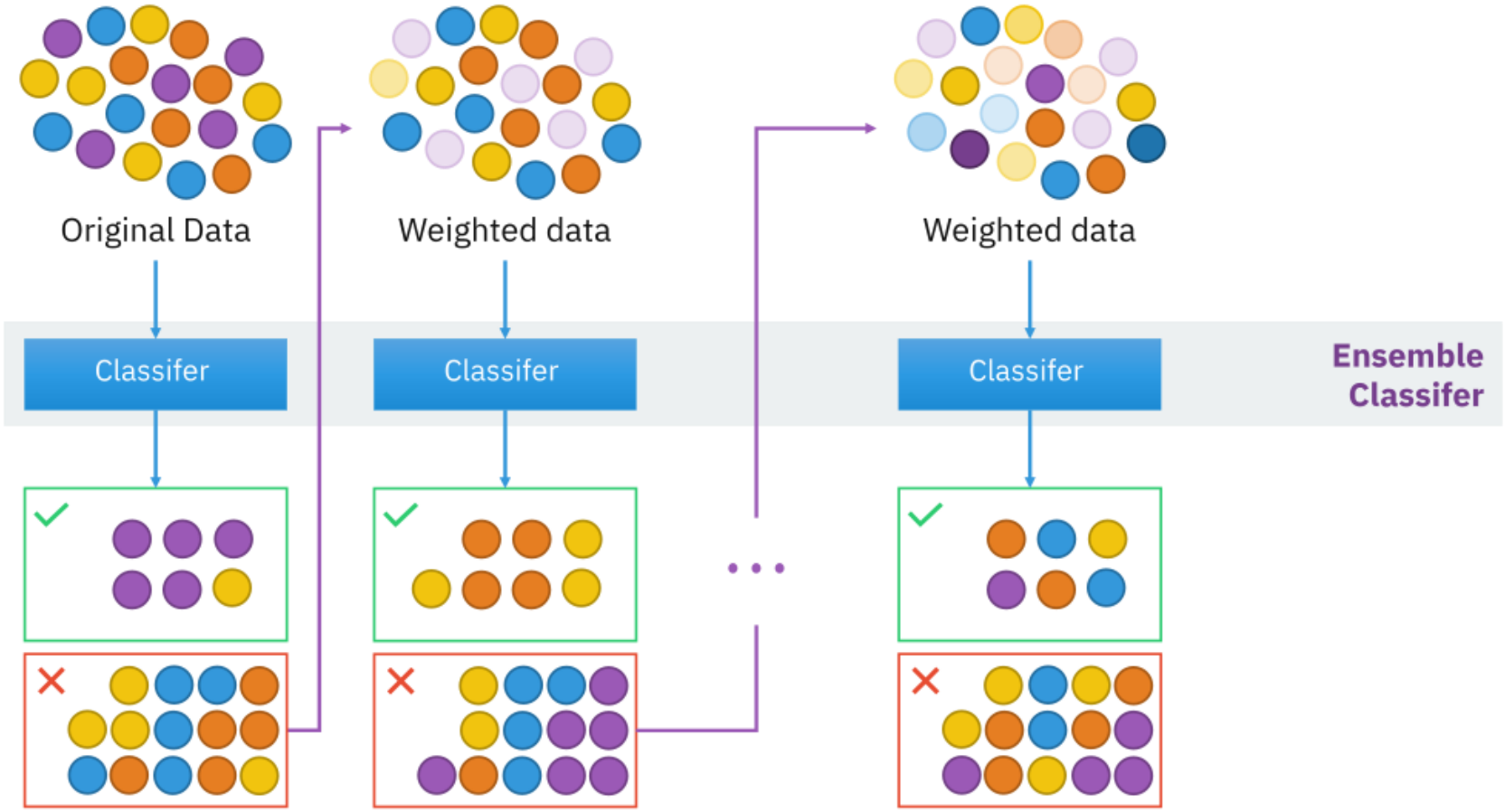
Variable importance plot for the `Heart` data

Outline

6.8 Boosting

- Each learner in this ensemble is trained on the same set of samples, but the samples are weighted differently among iterations.
- As a result, future weak learners focus more on the examples that previous weak learners misclassified.
- In boosting, each tree is grown *sequentially* using information from previously grown trees:
    1. You start by training the first weak classifier on the original dataset.
    2. Samples are reweighted based on how well the first classifier classifies them, e.g., misclassified samples are given higher weight.
    3. Train the second classifier on this reweighted dataset. Your ensemble now consists of the first and the second classifiers.
    4. Samples are weighted based on how well the ensemble classifies them.
    5. Train the third classifier on this reweighted dataset. Add the third classifier to the ensemble.
    6. Repeat for as many iterations as needed.
- Form the final strong classifier as a weighted combination of the existing classifiers—classifiers with smaller training errors have higher weights.

6.8 Boosting



6.8 Boosting

Boosting algorithm for regression trees

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \ldots, B$, repeat:
   - 2.1 Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r).
   - 2.2 Update $\hat{f}$ by adding in a shrunken version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

   - 2.3 Update the residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

3. Output the boosted model,

$$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x).$$

- $f(x)$ is the decision tree (model)
- r = residuals
- d = number of splits in each tree (controls the complexity of the boosted ensemble).
- λ = shrinkage parameter (a small positive number that controls the rate at which boosting learns; typically 0.01 or 0.001 but right choice can depend on the problem).

UMKC

6.8 Boosting

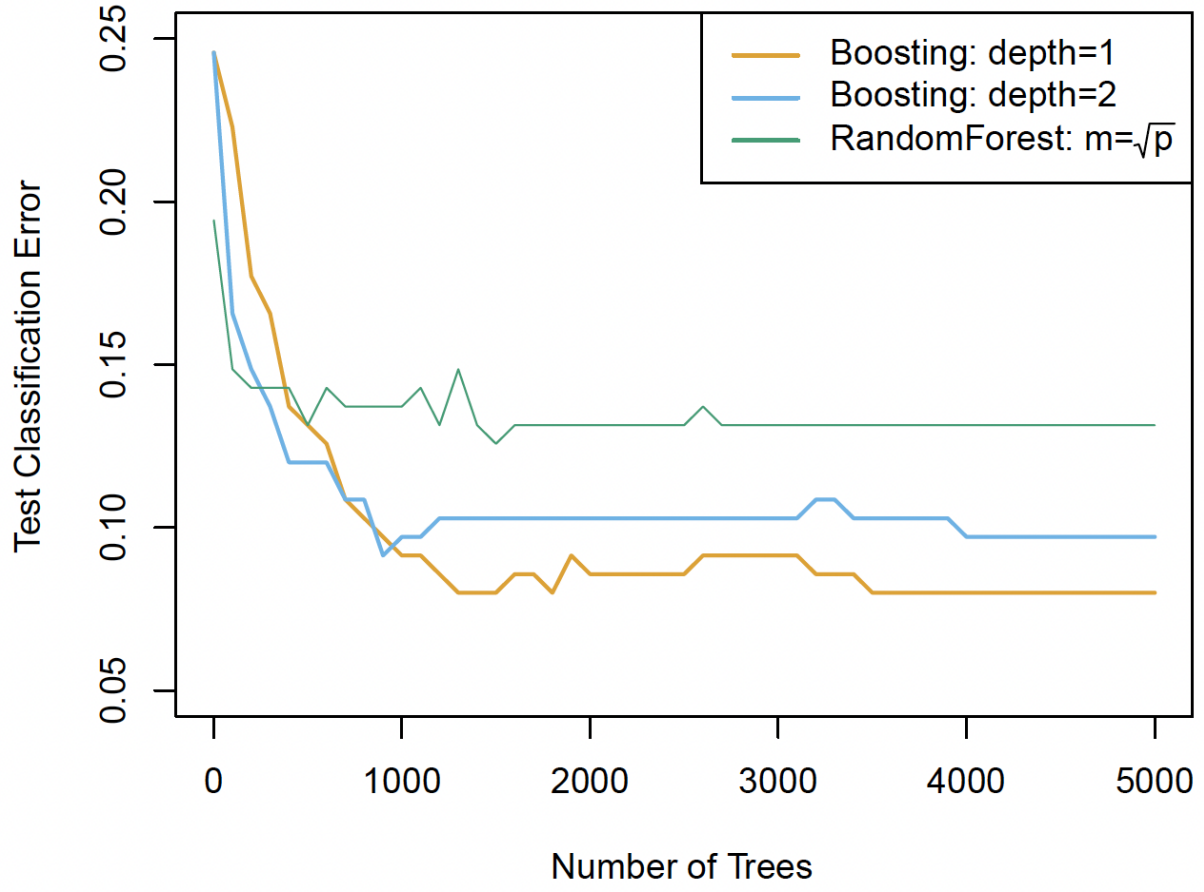
Boosting algorithm for regression trees – continued

- Each trees can be small, with just a few terminal nodes (determined by d).
- By fitting small trees to the residuals, we slowly improve our model ($\hat{f}$) in areas where it doesn't perform well.
- The shrinkage parameter λ slows the process down further, allowing more and different shaped trees to 'attack' the residuals.
- Unlike bagging and random forests, boosting can OVERFIT if B is too large.
- B is selected via cross-validation.
- Boosting for classification is similar in spirit to boosting for regression, but is a bit more complex. We will not go into the detail in this course.

6.8 Boosting

Boosting vs Random Forest

- Results from performing boosting and random forests on the 15-class gene expression data set in order to predict cancer versus normal.
- Depth-1 trees slightly outperform depth-2 trees, and both outperform the random forest, although the standard errors are around 0.02.
- Notice that because the growth of a particular tree takes into account the other trees that have already been grown, smaller trees are typically sufficient in boosting (versus random forests).



UMKC

Summary

- Decision trees are simple and interpretable models for regression and classification.
- However, they are often not competitive with other methods regarding prediction accuracy.
- Bagging, random forests, and boosting (also known as ensemble methods) are good methods for improving the prediction accuracy of trees.
- They work by growing many trees using the training data and then combining the predictions of the resulting ensemble of trees.
- Random forests and boosting are among the state-of-the-art methods for supervised learning.
- However, their results can be difficult to interpret.

Further Reading

- Chapter 8 – James G., Witten D., Hastie T., Tibshirani R., & Taylor J. (2023) *An Introduction to Statistical Learning with Applications in Python.* Springer https://link.springer.com/book/10.1007/978-3-031-38747-0

- Chapter 8 - Introduction to Statistical Learning Using R Book Club: https://r4ds.github.io/bookclub-islr/tree-based-methods.html

- Summary of Chapter 8: https://www.bijenpatel.com/guide/islr/tree-based-methods/

UMKC

End of Topic

Thank you

Any questions?

UMKC